

IIT Kharagpur



IIT Madras



IIT Goa



IIT PALAKKAD

IIT Palakkad

End to End Accelerated Deep Learning

Optimizing Deep Learning Training: Transfer Learning

Dr. Tosin Adesuyi
NVIDIA



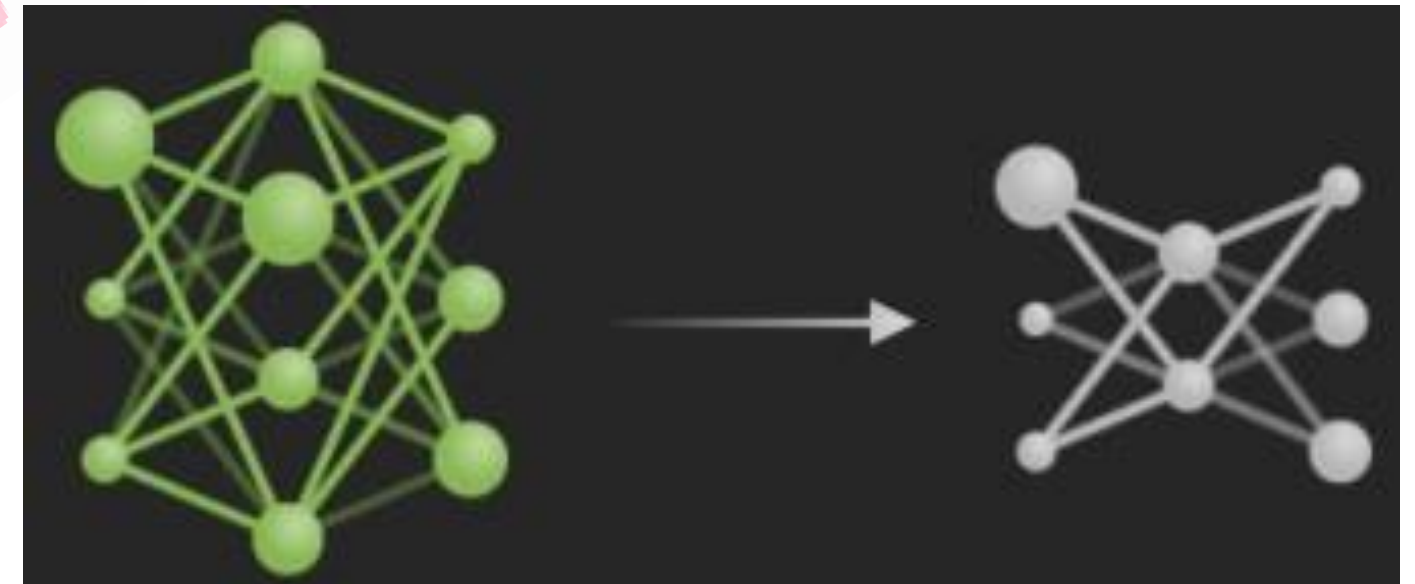
National
Supercomputing
Mission



Centre for
Development of
Advanced Computing

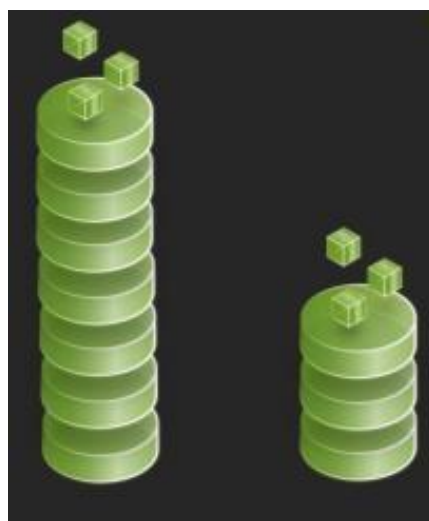
What is Transfer Learning ?

- **Transfer Learning** is a process of using learned features from a **pretrained model** to train new data. Thus, a new model is produced.
- The **new data** is trained using **weights** and **hyperparameters** from the pretrained model.
- The hyperparameters may be tune to fit desirable accuracy for the new model

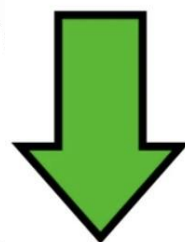


Why Transfer Learning ?

- You don't need to build your model from scratch



Required less data to train



High learning speed, therefore
training time is reduced



Save cost

Source: <https://blogs.nvidia.com/blog/2019/02/07/what-is-transfer-learning/>



Existing AI Domain for Transfer Learning

- Image Classification
- Object Detection
- Segmentation
- ASR (Automated Speech Recognition)
- NLP (Natural language Processing)

Computer Vision

Conversational AI



NPTEL

Transfer Learning With NVIDIA TAO

- TAO (Train, Adapt, Optimize)

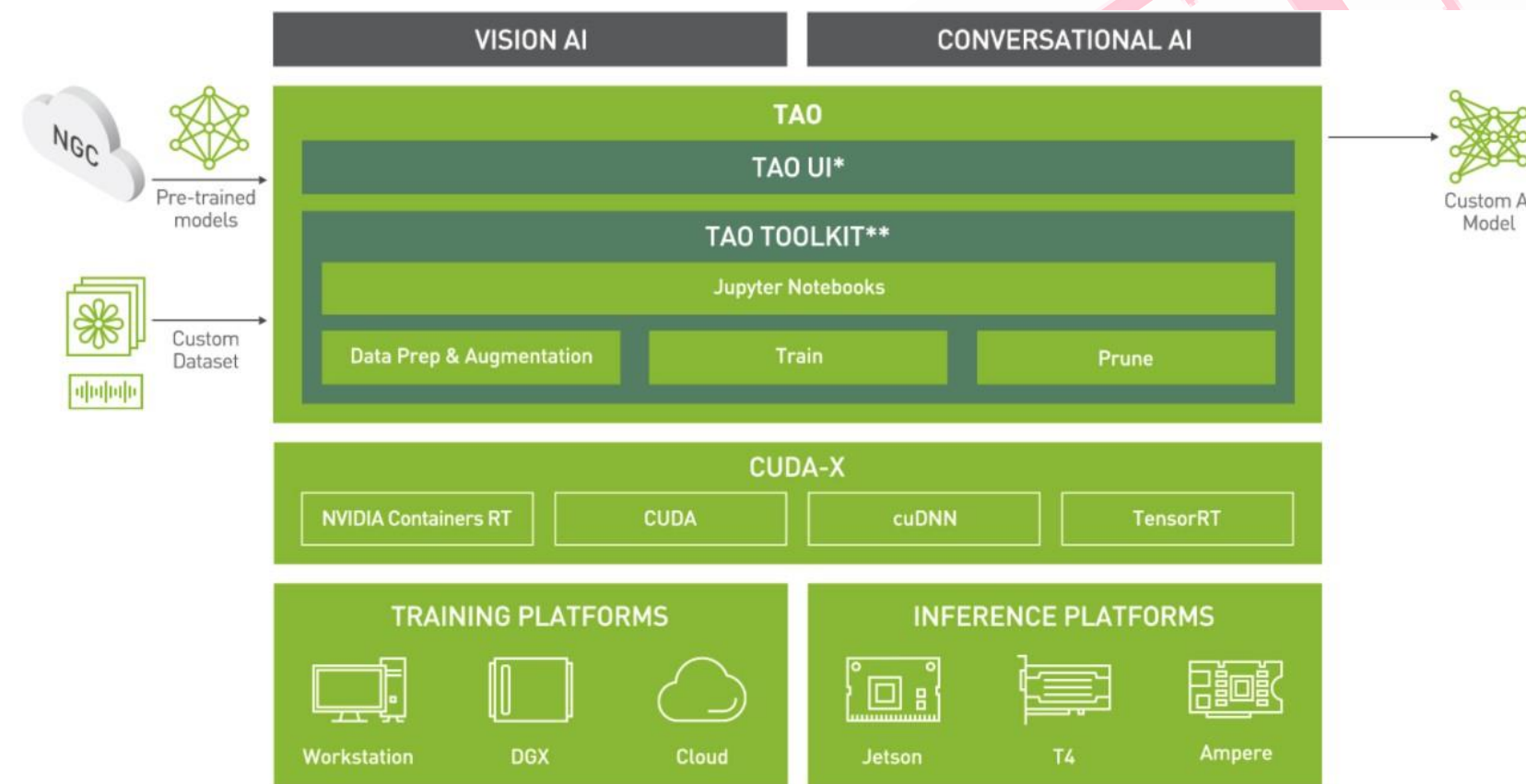
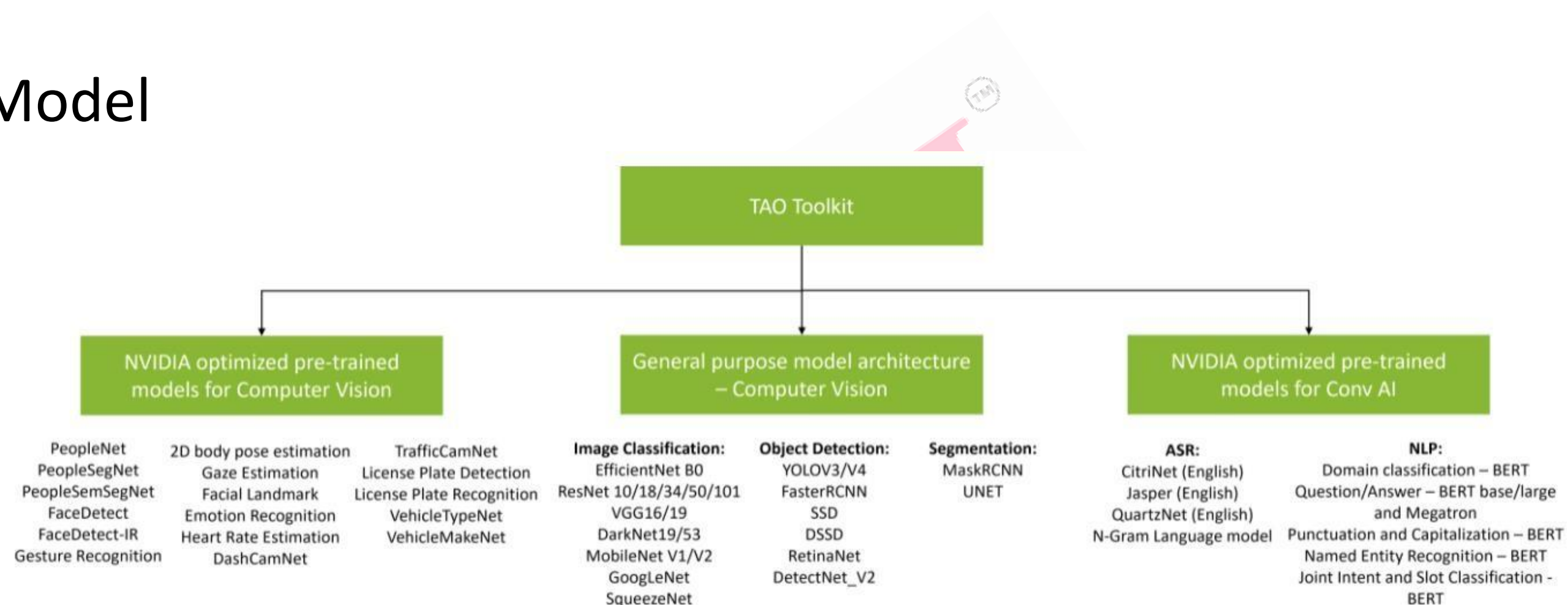


Image source: <https://developer.nvidia.com/tao-toolkit>

Transfer Learning With NVIDIA TAO

- Pretrain Model



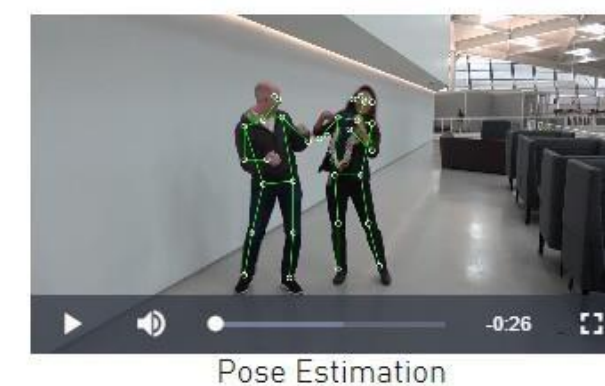
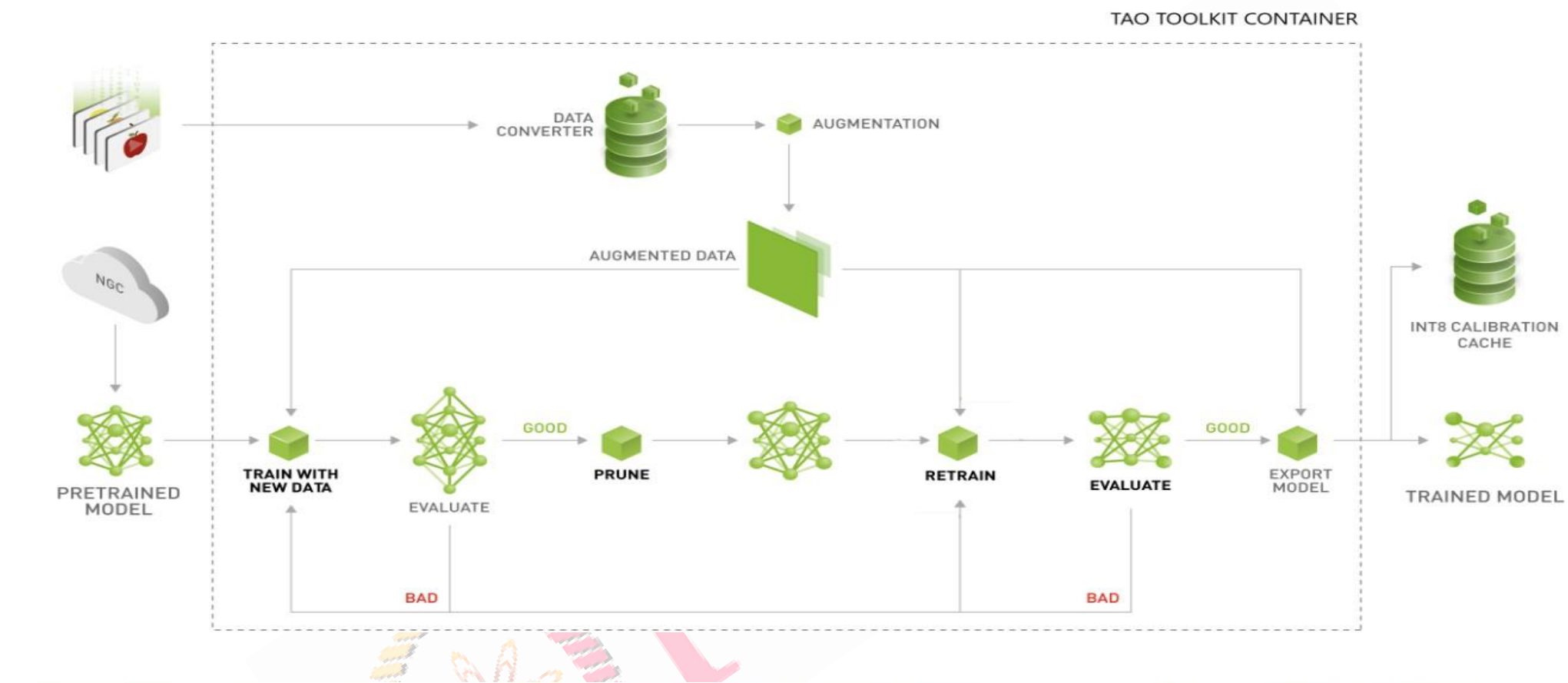
Two types of pre-trained models:

- Purpose-built pre-trained models
- General purpose models

Image source: <https://developer.nvidia.com/tao-toolkit>

TAO Toolkit

Computer Vision workflow

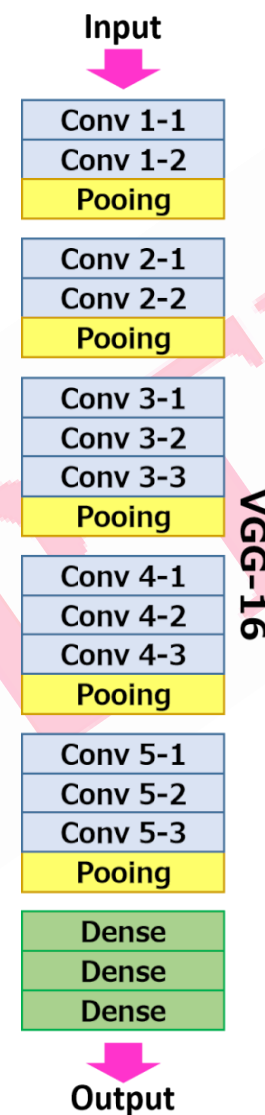


These AI models can be readily used for common **computer vision** use-cases

Image source: <https://developer.nvidia.com/tao-toolkit>

Transfer Learning: Image classification with VGG 16

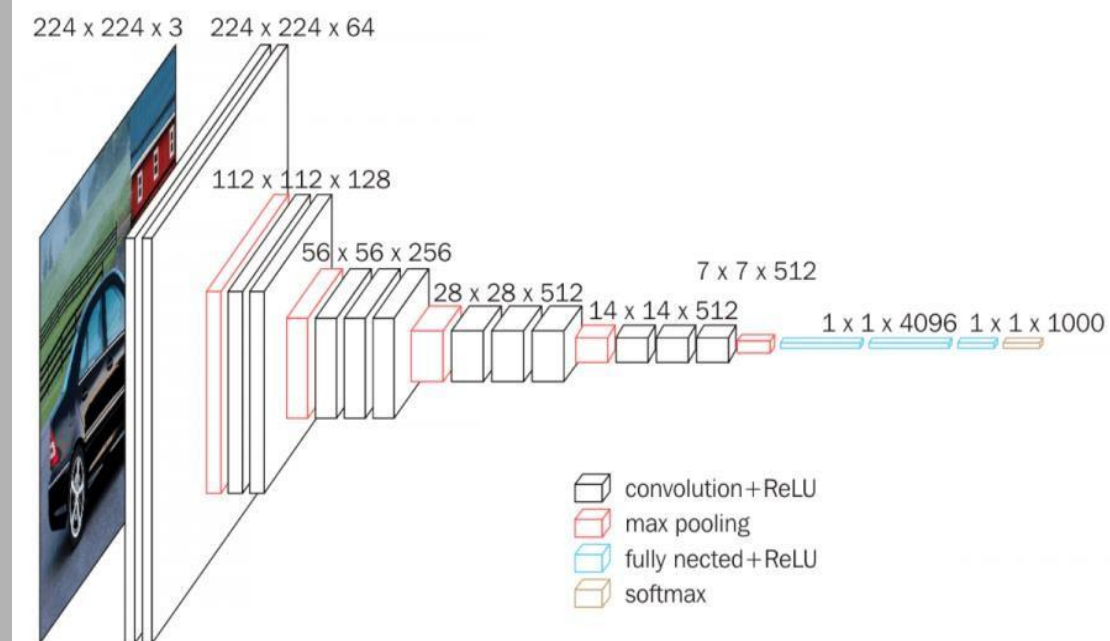
- In 2014 the VGG-16 network came in second place in the ILSVRC competition.
- It is a 16 layers network with a total of 138 Million parameters
- trained for two weeks on 4 NVIDIA GPUs
- Pretrained versions of the VGG-16 model are still often used as a feature extractor for other networks.



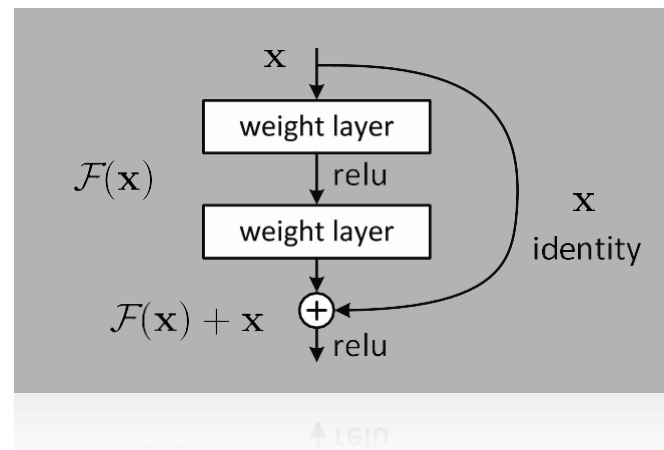
VERY DEEP CONVOLUTIONAL NETWORKS FOR LARGE-SCALE IMAGE RECOGNITION

Karen Simonyan* & Andrew Zisserman*

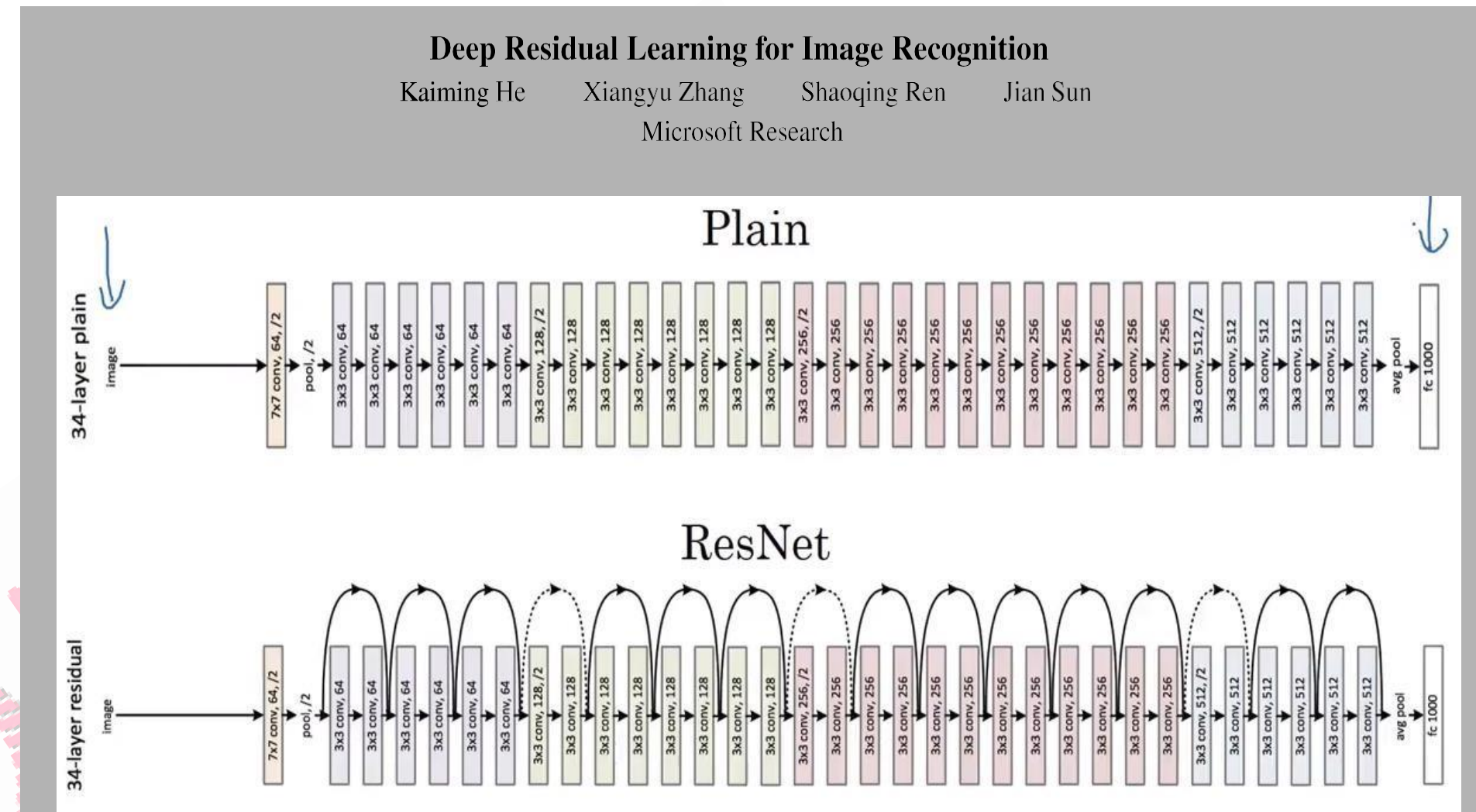
Visual Geometry Group, Department of Engineering Science, University of Oxford



Transfer Learning: Image classification with RESNET-50



<https://arxiv.org/pdf/1512.03385.pdf>



<https://arxiv.org/pdf/1409.1556.pdf>

- The ResNet-50 network was far deeper and more accurate than the VGG-16 network
- 2015 Microsoft Research. 50 Layers, 23M params.
- Skip connections were used liberally in the ResNet-50 architecture that won the 2015 ImageNet competition



Application Focus

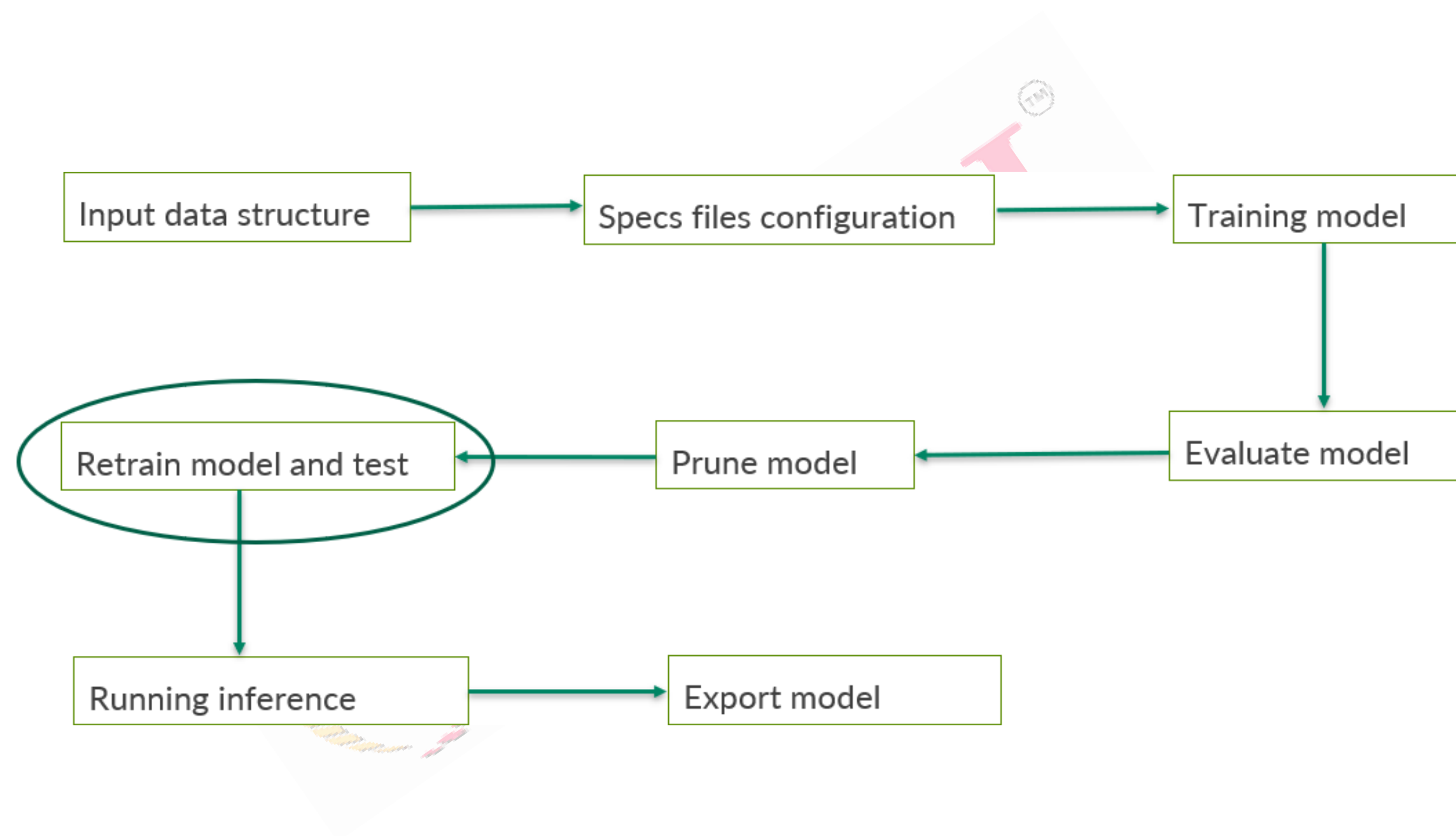
☒ NVIDIA TAO Toolkit

☐ YOLOv4

☒ YOLOv5

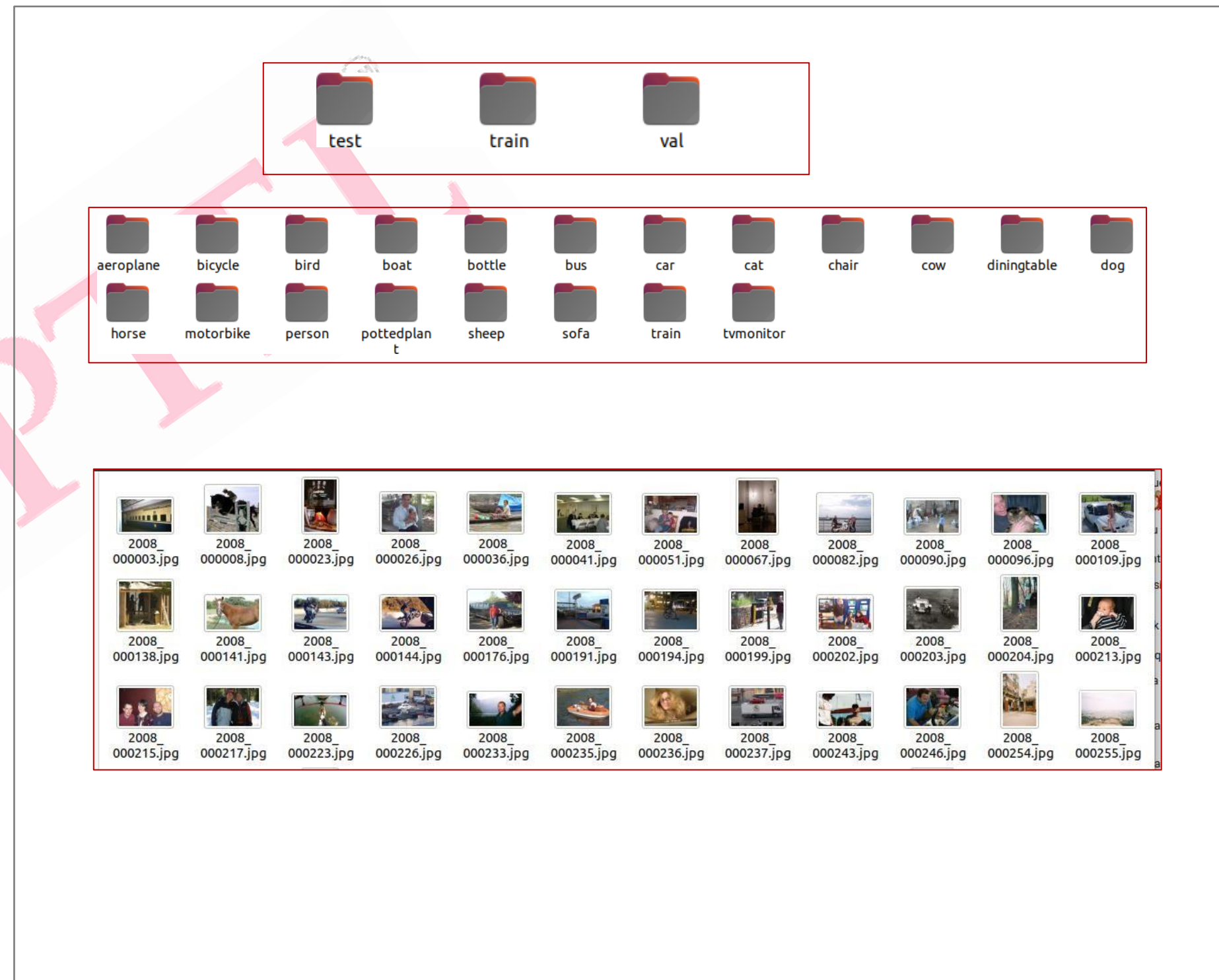


Overview of TAO image classification phase



TAO Application with Resnet 18 model

- Image Classification Task
- Input Dataset
 - ✓ Kitti
 - ✓ TFrecord

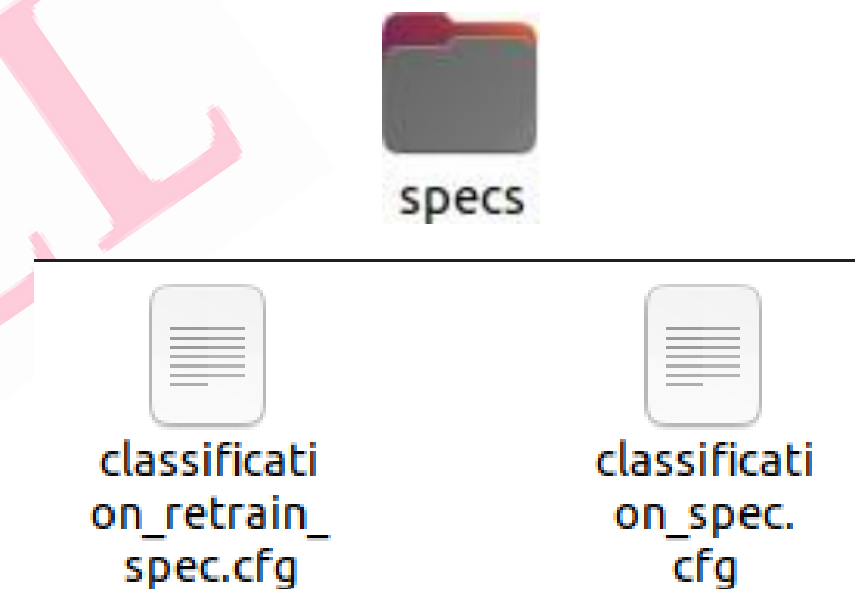




TAO Application with Resnet 18 model

Specification File for Classification

- The specs folder contains 2 specification files
 - ✓ Classification_spec.cfg
 - ✓ Classification_retrain_spec.cfg
- The specification file consists three main components:
 - ✓ Model_config
 - ✓ Train_config
 - ✓ Eval_config





classification_spec.cfg file

```
model_config {
  arch: "resnet",
  n_layers: 18
  # Setting these parameters to true to match the template downloaded from NGC.
  use_batch_norm: true
  all_projections: true
  freeze_blocks: 0
  freeze_blocks: 1
  input_image_size: "3,224,224"
}

train_config {
  train_dataset_path: "/workspace/tlt-experiments/data/split/train"
  val_dataset_path: "/workspace/tlt-experiments/data/split/val"
  pretrained_model_path: "/workspace/tlt-experiments/classification/pretrained_resnet18/tlt_pretrained_classification_vresnet18/resnet_18.hdf5"
  optimizer {
    sgd {
      lr: 0.01
      decay: 0.0
      momentum: 0.9
      nesterov: False
    }
  }
  batch_size_per_gpu: 64
  n_epochs: 80
  n_workers: 16
  preprocess_mode: "caffe"
  enable_random_crop: True
  enable_center_crop: True
  label_smoothing: 0.0
  mixup_alpha: 0.1
  # regularizer
  reg_config {
    type: "L2"
    scope: "Conv2D,Dense"
    weight_decay: 0.00005
  }
  # learning_rate
  lr_config {
    step {
      learning_rate: 0.006
      step_size: 10
      gamma: 0.1
    }
  }
}

eval_config {
  eval_dataset_path: "/workspace/tlt-experiments/data/split/test"
  model_path: "/workspace/tlt-experiments/classification/output/weights/resnet_080.tlt"
  top_k: 3
  batch_size: 256
  n_workers: 8
  enable_center_crop: True
}
```



classification_retrain_spec.cfg file

```
Open  ▾  [+]
```

```
classification_retrain_spec.cfg
~/tlt-experiments_01/specs

model_config {
  arch: "resnet",
  n_layers: 18
  use_batch_norm: true
  all_projections: true
  input_image_size: "3,224,224"
}
train_config {
  train_dataset_path: "/workspace/tlt-experiments/data/split/train"
  val_dataset_path: "/workspace/tlt-experiments/data/split/val"
  pretrained_model_path: "/workspace/tlt-experiments/classification/output/resnet_pruned/resnet18_nopool_bn_pruned.tlt"
  optimizer {
    sgd {
      lr: 0.01
      decay: 0.0
      momentum: 0.9
      nesterov: False
    }
  }
  batch_size_per_gpu: 64
  n_epochs: 80
  n_workers: 16
  preprocess_mode: "caffe"
  enable_random_crop: True
  enable_center_crop: True
  label_smoothing: 0.0
  mixup_alpha: 0.1
  # regularizer
  reg_config {
    type: "L2"
    scope: "Conv2D,Dense"
    weight_decay: 0.00005
  }
  # learning_rate
  lr_config {
    step {
      learning_rate: 0.006
      step_size: 10
      gamma: 0.1
    }
  }
}
eval_config {
  eval_dataset_path: "/workspace/tlt-experiments/data/split/test"
  model_path: "/workspace/tlt-experiments/classification/output_retrain/weights/resnet_080.tlt"
  top_k: 5
  batch_size: 256
  n_workers: 8
  enable_center_crop: True
}
```



DEMO





YOLOv5

- **Yolov5** is a family of object detection architectures and models pretrained on the COCO dataset

<https://github.com/ultralytics/yolov5>

The image shows a screenshot of the YOLOv5 GitHub repository page on the left and the YOLOv5 mobile app interface on the right. The GitHub page displays the repository name 'ultralytics / yolov5', statistics (229 issues, 27 pull requests), and a list of recent commits. The mobile app interface shows the YOLOv5 logo, a list of detected objects (car, person, etc.) with bounding boxes, and download links for the App Store and Google Play.

GitHub Repository Details:

- Repository: `ultralytics / yolov5` (Public)
- Stats: 229 Issues, 27 Pull requests, 283 Watchers, 8.3k Forks
- Commits: 1,850
- Recent Commits:
 - glenn-jocher Update TQDM bar format (#6988) - 10 hours ago
 - Bump actions/checkout from 2 to 3 (#6881) - 9 days ago
 - Update hyp.VOC.yaml (#6772) - 19 days ago
 - YOLOv5 Export Benchmarks for GPU (#6963) - yesterday
 - Update TQDM bar format (#6988) - 10 hours ago

YOLOv5 Mobile App Interface:

- YOLOv5 by Ultralytics
- Detected objects: car, person, etc.
- Download on the App Store / Coming Soon on Google Play
- CI CPU testing: passing
- DOI: 10.5281/zenodo.6222936
- Docker pulls: 204k
- Open in Colab, Open in Kaggle, Slack, Join Forum

DOWNLOAD YOLOV5:

```
git clone https://github.com/ultralytics/yolov5
cd yolov5
pip install -r requirements.txt
```

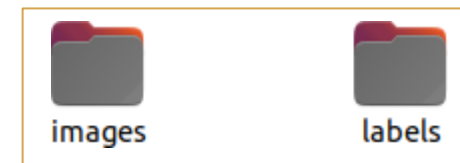
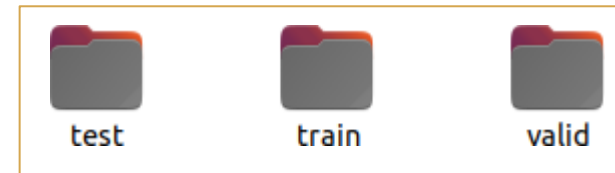
HOW TO RUN YOLOv5:

```
python detect.py --source 0 # webcam
img.jpg # image path
vid.mp4 # video path
path/ # directory path for set of images
'https://youtu.be/Zgi9g1ksQHc' # YouTube
'rtsp://example.com/media.mp4' # RTSP, RTMP, HTTP stream
```




Custom data training with YOLOv5

- Prepare your dataset



Object-class X-center Y-center Width height

1	0.3421875	0.46875	0.265625	0.9319444444444445
0	0.53125	0.7555555555555555	0.18333333333333332	0.46111111111111114

- **Annotate/label** your dataset using **Roboflow** (<https://roboflow.com/?ref=ultralytics>). You may as well use Yolo_mark

- Prepare your **data.yaml** file



```
path: ../yolov5/dataset/  
train: train/images  
val: valid/images  
  
nc: 2  
names: ['dog', 'person']
```



- Select weight based on the YOLOv5 model size preference (<https://github.com/ultralytics/yolov5/releases>)



▼ Assets 12

- yolov5l.pt
- yolov5l6.pt
- yolov5m.pt
- yolov5m6.pt
- yolov5n.pt
- yolov5n6.pt
- yolov5s.pt
- yolov5s6.pt
- yolov5x.pt
- yolov5x6.pt

- **Train your custom data**

```
python3 train.py --img 640 --data ../yolov5/dataset/data.yaml --cfg models/yolov5l.yaml --weights yolov5l.pt --batch 64 --epochs 100
```

- **Run inference**

```
python3 detect.py --source 0 --weights ../yolov5/runs/train/exp6/weights/best.pt --img 640
```

Source: <https://github.com/ultralytics/yolov5/wiki/Train-Custom-Data>



DEMO



Thank You